VpConfluence: completeness-weighted developing profiles (spec, rev 2: decisions applied)

Rev	Date	Change
1	2026-10-30	Draft for review
2	2026-10-30	Ettienne Hugo decisions: Time on by default; developing sources count at w >= 0.67; half days accept lower weight; Linear ramp, start 0.10, min 0.10; weight scales nested REFINEMENT bonus. NoRef 0.50 still open.

Date: 2026-10-30 (rev 2). Base: `origin/main` @ `93ff256` (includes PR #47 `55b4840`). Scope: `_VpConfluenceEngine.Function.CS`, `_VolumeProfileBase.Strategy.CS` (config + zone fields), `tests/VpHelperTests/Program.cs`. PriceAction library unchanged. One PR, separate from anything else.

1. Problem

A developing (incomplete) profile is tagged `developing = !extra.pocComplete` (engine L351) and today its labels score at full credit (`makeSurvivor`, L705-723): a POC built from 20 minutes of a session weighs the same as a finished one. Unless `assignDevelopingSources` (L501-511) remaps it onto a completed parent's source, it also counts as its own source in `independentSources` (`ZoneSourceCounts`, L89-101), and that drives `_DeltaDivRevFuturesLELX` target ranking (strategy L554) and base zone ordering (base L2306).

2. What the code has (no new inputs needed)

- `Anchor.windowStart / windowEnd` (L210-211) come from `barTime(ds, startBar/endBar)` (L354-355). For a developing profile `endBar` is clipped to the current bar (L349-350), so **the expected end of the window is not known** anywhere in the engine. The total has to be estimated.
- `confluenceLookbackExtras(ds)` (base L2359) returns age 0 (possibly developing) plus up to `ConfluenceLookbackProfiles - 1` (default 4) earlier profiles **on the same stream and resolution**. Those are the natural reference: their bar counts (`endBar - startBar + 1`) and volumes (`ShareSnap.total`, L885-892) are already read.
- Every stream has a fixed MC resolution (session, weekly, ...), so "same stream, older profiles" means "same window type". The engine builds no composites (L184-185); a composite fed in as a stream gets the same treatment.

3. Formula

For a non-overlay anchor of profile P on stream ds:

```
w_complete = 1                                                  if P is completed (unchanged
raw        = elapsed(P) / expected(P)        clamped to [0, 1]
r          = ramp(raw):  Linear -> raw ;  Sqrt -> sqrt(raw)
w_complete = 0                 if raw < DevStartFraction    (hit listed, credit 0)
           = max(DevMinWeight, r)   otherwise
credit'    = credit x w_complete
```

Time basis (default). `elapsed = P.endBar - P.startBar + 1` (stream bars so far). `expected = median bar count of the completed lookback profiles on ds`. Counting bars rather than wall-clock time copes with overnight gaps in weekly windows and weekends. If there is no completed reference (start of the chart), use `w = DevNoRefWeight`.

Volume basis (option). `elapsed = snap.total(P)` and `expected = median snap.total of the completed lookback profiles`. Strengths: it tracks how much auction actually happened, and on a half day it gets close to "complete" faster because volume is front-loaded. Weaknesses: volume is U-shaped within a session (the open is heavy, so it overstates early), a 4-profile median is noisy around rolls and event days, and `snap.total` is missing when levels are not read (falls back to `DEFAULT_CONFLUENCE_SHARE_PIVOT` paths). **Recommendation: Time is the default and Volume is an enum option.**

Max(time, volume) is offered as a third mode ("Either") for review. It is not the default.

Multi-timeframe. The reference is always the same stream, so session windows compare to sessions and weekly to weeks. The rank weight `ConfluenceWeightN` still multiplies first, so `credit = rankWeight x shareFactor x naked x w_complete`.

Half days. With the time basis, the prior-session median overstates `expected`, so `w` stays low for the whole short session. That errs conservative and never reaches 1 before close. Once the profile closes, `pocComplete` sets w = 1, so nothing gets stuck. A precise fix (scale `expected` by the RTH length from `_EventCalendar.isHalfDay`) would couple the engine to the calendar. Not proposed for v1. **DECIDED (Q3): accept the lower weight, no calendar dependency.** The median ignores a single prior half day automatically. Partial profiles (`isPartial`, the window began before the first bar) are excluded from the reference set.

Nesting. `applyNested` (L780-785) sets a refined child to `REFINEMENT x parent.credit`. The parent is completed (`completedOnly`), so its credit is unchanged. The developing child's refinement credit is then multiplied by its own `w_complete`. Apply w in one place, after `applyNested`, to the survivor's `credit`, with `baseCredit` keeping the unweighted value.

4. independentSources: recommendation

Options: (a) completed profiles only; (b) completed, plus a developing source once `w >= DevSourceThreshold`. **DECIDED (Q2): (b) with `DevSourceThreshold = 0.67` by default: a developing profile counts once in independentSources when `w_complete >= 0.67` (two thirds complete).** This removes the "half an hour of POC adds a whole source" inflation in LELX ranking; setting 1.01 restores never-count. Implementation: `IVpConfSurvivor` gains `double Completeness { get; }` (1 for completed and overlays). `ZoneSourceCounts` skips a survivor with `Developing && Completeness < threshold` (a new `threshold` argument). `profileCount` follows the same rule.

`developingHits` / `developingScore` stay separate (PR #47 contract). `developingScore` = sum of the *weighted* credits. Each developing `ConfluenceHit` also carries `completeness` and `rawCredit`.

5. Config (protected, base; no `[Input]`)

Field	Type	Default	Meaning
`ConfluenceDevWeighting`	enum `EDevWeighting {Off, Time, Volume, Either}`	`Time` (DECIDED, on by default)	Off = today's behaviour (w = 1)
`ConfluenceDevRamp`	enum `{Linear, Sqrt}`		shape of r

Field	Type	Default	Meaning
		`Linear` (DECIDED)	
`ConfluenceDevStartFraction`	double [0,1]	0.10 (DECIDED)	below this, credit 0
`ConfluenceDevMinWeight`	double [0,1]	0.10 (DECIDED)	floor once started
`ConfluenceDevNoRefWeight`	double [0,1]	0.50 (OPEN)	no completed reference available
`ConfluenceDevSourceThreshold`	double [0,1.01]	0.67 (DECIDED)	developing counts as a source at w >= this

Each one is validated with `requireAtLeast`-style setters (it throws if out of range), just like `ConfluenceSplitAtrFraction` (base L626). Constants go in `VpConfluenceRules` (`DefaultDev*`).

6. Run only when used

All work happens inside `Rebuild()`, which only runs when the confluence engine exists (`m_confluenceEngine` created at base L2331, behind confluence use). Medians are computed once per stream per rebuild, and only if some developing profile exists on that stream. No allocation is made when `ConfluenceDevWeighting == Off` or there is no developing extra. No `NeedsDaily` / calendar dependency is added.

7. Code touch points (origin/main line refs)

1. `VpConfluenceRules` (L24-150): add `CompletenessWeight(double elapsed, double expected, ramp, start, min, noRef)` (pure) and `Median(IList<double>)`. Extend `IVpConfSurvivor` (L26) with `Completeness`. `ZoneSourceCounts` (L89) gains a `devSourceThreshold` argument. `DevelopingHits` (L121-140) is unchanged in logic because it sums the already weighted `Credit`. Update its doc comment.
2. `Anchor` (L191-212): add `completeness` (default 1) and `elapsedBars` / `volTotal`.
3. `collectStream` (L331-344): after the loop, compute the reference medians from the completed, non-partial extras. `addProfile` (L346-372) sets `completeness` on developing anchors.
4. `addZone` (L560-565): after `applyNested(survivors)`, do `s.credit *= s.anchor.completeness` for developing survivors (one place).
5. `addZone` L626: pass the threshold to `ZoneSourceCounts`. L629 is unchanged.
6. `toHit` (L823-840) / `ConfluenceHit` (base L336): add `completeness` and `rawCredit` (= baseCredit).
7. Base L600-640: the six protected properties plus the enums.
8. Untouched: `EmitZones` (L106) / `SplitAt` (L47), because clustering is by price and weighting does not move anchors. `collapseEqualWindows` and `assignDevelopingSources` are also untouched.

8. Journal

When journal is on (`confluenceJournal`), print one line per developing profile per rebuild: `VPCONF devweight ds(2) profile(ds2:s1234) basis(time) elapsed(37) expected(81) raw(0.457) ramp(linear) w(0.457) refN(3) source(no|yes)`. Use `basis(noref)` when there is no reference, and `w(0) start(0.10)` when below the start fraction. `source(yes)` when w >= the threshold (default 0.67), else `source(no)`. Add w= to the existing `_VolumeProfileTest1` zone trace (L420).

9. Tests (`tests/VpHelperTests/Program.cs`)

Behaviour: 1. Completed survivors: credit, rawScore, independentSources identical to the `Off` mode (fixture from the existing ZoneSourceCounts cases, L494-502). 2. Linear: elapsed 40 / expected 80 gives w = 0.5. Sqrt gives 0.7071. 3. raw < start gives credit 0, while the hit is still listed in developingHits. 4. Floor: raw 0.12, min 0.25 gives 0.25. 5. raw > 1 (a long session) clamps to 1. 6. No reference gives the noRef weight. 7. Median ignores one short (half-day) prior profile. Partial profiles are excluded. 8. Volume basis: 300 / 1000 gives 0.3. Either = max. 9. Weekly stream references weekly profiles only (two streams, different medians). 10. Nested developing child: credit = 0.25 x parent x w (the weight scales the REFINEMENT bonus, DECIDED Q5). 11. Default threshold 0.67: a developing own-source survivor with w 0.66 does not count in independentSources, w 0.67 counts (boundary), w 0.80 counts once. Threshold 1.01: never counts. Threshold 0.5 with w 0.6 does count, and w 0.4 does not. 14. Default config: `ConfluenceDevWeighting == Time`, `ConfluenceDevRamp == Linear`, start 0.10, min 0.10, `ConfluenceDevSourceThreshold == 0.67`. 12. developingScore = sum of weighted credits, and developingHits stays null with no developing anchor. 13. `Off`: byte-identical zone list to main on a fixture.

Mutation resistance (in the existing `pin1` / `count` source-pin style, L542-557): - Use asymmetric values (elapsed 30 / expected 80, ramp sqrt, min 0.2) so swapped operands, linear-vs-sqrt, min-vs-max, or a missing clamp each fail a different assert. - Boundary checks at exactly raw == start (credit > 0) and w == threshold (counts), which catch < vs <= mutations. - Pin: `s.credit *= s.anchor.completeness` appears exactly once and after `applyNested(survivors);`. - Pin: `ZoneSourceCounts(survivors, m_owner.ConfluenceDevSourceThreshold, ...)`. - Assert the weight is never applied to overlays (VWAP context score unchanged). - Setter range checks throw.

10. Comparability / pre-registration

With the default `Time`, scores **change** for every strategy whose zones contain developing labels. `rawScore`, `acceptanceScore`, `zScore` and ordering shift, and `independentSources` can drop by 1 when a developing own source below w 0.67 is no longer counted. `_DeltaDivRevFuturesLELX` ranks on `independentSources` (L554), so its targets and backtests are not comparable with pre-change runs. Proposal: record the PR hash and config values in the run journal header. Re-run the LELX baseline with `ConfluenceDevWeighting = Off` (it must match main exactly, see test 13) and with `Time` before evaluating, and pre-register the comparison metric (target hit rate / avg R) before looking at the `Time` results. No parameter tuning on the same sample.

11. Decisions (Ettienne Hugo, 2026-10-30)

1. Default: **DECIDED.** `Time`, on by default (not `Off`). Pre-registration and comparability note (section 10) kept, including the baseline re-run with `Off` matching main.
2. independentSources: **DECIDED.** Developing profiles count once `w_complete >= 0.67`; `ConfluenceDevSourceThreshold` default 0.67 (tests 11, 14).

3. Half days: **DECIDED.** Accept the lower weight; no calendar dependency.
4. Ramp: **DECIDED.** `Linear`; start 0.10 and min 0.10. **OPEN:** noRef weight 0.50 not yet confirmed.
5. Nested REFINEMENT bonus: **DECIDED.** Yes, the developing weight scales a nested developing child's 0.25 bonus (test 10).